----------Framework----------

What is framework ?

A framework is a pre-built structure or set of tools that helps developers build software applications faster and in an organized way.

Framework = A ready-made structure where you add your own code to build an application.

Think of it like building a house :

- **Without a framework → you build everything from the ground up.**
- **With a framework → the foundation, structure, and many tools are already provided.**

Framework	Mainly used for
Flask	Web apps & REST APIs
Django	Full web applications
FastAPI	High-performance APIs
PyTorch	Machine learning/deep learning
TensorFlow	Machine learning/deep learning

Framework vs Library

Library:

Your Code → Library

You call the library when you need it.

Framework:

Framework → Your Code

The framework controls the overall application flow and calls your code at the appropriate points.

A framework provides a predefined architecture, tools, and rules that help you develop applications faster, consistently, and with less repetitive code.

What is Flask?

Flask is a lightweight Python web framework used to build:

- **Web applications**
- **REST APIs**
- **Backend systems**
- **Microservices**
- **CRUD applications**
- **Authentication systems**
- **Database-driven applications**

Flask is called a micro-framework because it gives you the core functionality and lets you add other components when needed.

Flask architecture

A simple Flask application works like:

Client / Browser / Postman

↓

HTTP Request

↓

Flask App

↓

Route

↓

View Function

↓

Business Logic

↓

Database

↓

HTTP Response

↓

Client / Browser / Postman

Prerequisites

Before Flask, you should understand these Python concepts:

Python basics

- **Variables**
- **Data types**
- **Operators**
- **Conditions**
- **Loops**
- **Functions**
- **Lists**
- **Tuples**
- **Sets**
- **Dictionaries**
- **Strings**
- **Exception handling**
- **Modules and packages**
- **Classes and objects**
- **OOP basics**
- **Virtual environments**

What is HTTP?

HTTP = HyperText Transfer Protocol

It is the communication system used between a client and a server.

Think of a restaurant:

You (Client)

↓

Order

↓

Restaurant (Server)

↓

Prepare order

↓

Response

↓

You (Client)

In web development:

Browser / Postman

↓

HTTP Request

↓

Flask Server

↓

Python Code

↓

Database

↓

HTTP Response

↓

Browser / Postman

GET — "Give me the data"

Imagine you go to a school office and say:

"Please give me the list of all students."

You are asking for information, not changing anything.

That's GET.

GET /students

POST — "Create something"

Now you go to the school office and say:

"Please register a new student."

You're creating new data.

That's POST.

PUT — "Replace/update the complete data"

PUT = Replace/update the resource

PATCH — "Change only part of the data"

DELETE — "Remove something"

You go to the school office:

"Remove student #10 from the system."

Remember CRUD + HTTP

This is extremely important for backend development.

CRUD	HTTP	Meaning
Create	POST	Create data
Read	GET	Read data
Update	PUT/PATCH	Update data
Delete	DELETE	Delete data

What is a Request?

A request is what the client sends to the server.

For example:

GET /students

The browser/Postman is saying:

"Server, please give me the students."

A request can contain:

Method

URL

Headers

Body

What is a Response?

The response is what the server sends back.

Example:

Request:

GET /students

Response:

```
{
  "students": [
    "Rahul",
    "Amit",
    "Shashank"
  ]
}
```

Client → Request → Server

Server → Response → Client

What are Headers?

Headers contain additional information/metadata about the request or response.

Think of sending a parcel 📦.

The parcel has:

Name

Address

Type

Special instructions

HTTP headers work similarly.

Example:

Content-Type: application/json

This tells the server:

"The data I'm sending is JSON."

This tells the server:

"The data I'm sending is JSON."

Another common header:

Authorization: Bearer <token>

This tells the server:

"Here is my authentication token."

Example request:

POST /students

Content-Type: application/json

Authorization: Bearer abc123

What is the Body?

The body contains the actual data being sent.

For example, when creating a student:

```
{
  "name": "Shashank",
  "age": 22,
```

"course": "MTech"

}

That's the request body.

That's the request body.

Think:

Headers → Information about the request

Body → Actual data

For GET requests, you commonly don't send a request body; query parameters are often used instead.

----------What is a Status Code?

The server uses a status code to tell the client what happened.

200 — OK

201 — Created

400 — Bad Request

401 — Unauthorized

403 — Forbidden

404 — Not Found

500 — Internal Server Error

What is a URL?

URL = Uniform Resource Locator

It's the address used to locate a resource.

Example:

https://example.com/students/10

Break it down:

https://

↓

Protocol

example.com

↓

Domain

/students/10

↓

Path

```
@app.route("/students/10")
def student():
    ...
```

What is an Endpoint?

An endpoint is a specific API location through which a client interacts with your application.

For example:

GET /students

POST /students

GET /students/10

PUT /students/10

PATCH /students/10

DELETE /students/10

These are different API operations/endpoints.

Notice something important:

GET /students

POST /students

They use the same path but different HTTP methods, so they perform different operations.

What is JSON?

JSON = JavaScript Object Notation

It's a common format for exchanging data between frontend and backend.

Example:

```
{
  "name": "Shashank",
  "age": 22,
  "course": "MTech"
}
```

Think of JSON as a structured way of saying:

Name → Shashank

Age → 22

Course → MTech

JSON is extremely common in Flask REST APIs.

The most important flow to memorize

CLIENT

↓

REQUEST

↓

METHOD + URL + HEADERS + BODY

↓

FLASK

↓

ROUTE

↓

PYTHON LOGIC

↓

DATABASE

↓

RESPONSE

↓

STATUS CODE + HEADERS + BODY

↓

CLIENT

Installing Flask

Create a project:

mkdir flask_project

cd flask_project

Create virtual environment:

python -m venv venv

Activate on Windows:

venv\Scripts\activate

Install Flask:

pip install flask

Check:

pip show flask

Your First Flask Application

Create:

app.py

ex –

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Hello Flask!"
```

```
if __name__ == "__main__":
    app.run(debug=True)
```

```
python app.py
```

ex-

```
@app.route("/about")
def about():
    return "About Page"
```